

Лабораторная работа №4

Средства языка SQL для выборки данных

Цель работы: изучить основные принципы создания SQL-запросов для выборки данных из таблиц базы данных и представления данных в требуемом виде.

Основные теоретические сведения

Одной из основных задач, выполняемых на сервере баз данных, является выборка данных, их первичная обработка и представление в удобном виде. Для получения данных из таблиц базы данных используются SELECT-запросы, которые состоят из нескольких секций, большинство из которых могут отсутствовать.

SELECT-запросы в общем случае имеют следующий вид:

```
SELECT [TOP top_N] [DISTINCT] field_value[, field_value, ...]  
[FROM table_source]  
[WHERE condition]  
[GROUP BY field_name[, field_name, ...]  
[HAVING condition]]  
[ORDER BY field_name [sort_type][, field_name [sort_type], ...]
```

Фактический порядок выполнения секций SELECT-запроса отличается от порядка их следования в тексте запроса.

Секция FROM. В секции FROM указывается таблица данных, из которых выбирается необходимая информация. Это может быть физическая таблица базы данных, соединение нескольких таблиц или табличное выражение (будет рассмотрено в разделе «Вложенные подзапросы»).

В языке SQL существуют операции соединения двух таблиц, которые имеют следующий вид:

```
table_1 [[AS] alias_1] [join_type] table_2 [[AS] alias_2]  
[ON condition]
```

где table_1 и table_2 — таблицы или табличные структуры, alias_1 и alias_2 — псевдонимы таблиц, которые будут использованы в текущем запросе вместо имени таблицы или в качестве имени табличного выражения, если они указаны, join_type — тип соединения, condition — условие соединения таблиц. Результатом операции соединения двух таблиц является набор данных — табличная структура, содержащая все столбцы обеих таблиц.

Псевдонимы таблиц обязательны для табличных выражений, а также в случае, если выполняется операция соединения таблицы данных с этой же таблицей — для устранения неоднозначности их использования. В остальных

случаях использование коротких имён в качестве псевдонимов позволяет сократить объём текста SQL-запроса.

Существуют различные типы соединения таблиц: CROSS JOIN (перекрёстное соединение), INNER JOIN (внутреннее соединение), LEFT JOIN или LEFT OUTER JOIN («левое» соединение), RIGHT JOIN или RIGHT OUTER JOIN («правое» соединение, FULL JOIN или FULL OUTER JOIN (полное соединение). Подробно о каждом типе соединения ознакомьтесь самостоятельно. Перекрёстное соединение является единственным типом соединения, для которого не задаётся секция ON. Для всех остальных типов условие соединения таблиц является обязательным.

В результате выполнения секции FROM создаётся временная таблица данных, в которой имена и типы столбцов соответствуют столбцам заданных таблиц и табличных структур, а строки представляют результирующий набор данных.

Секция WHERE. В секции WHERE указывается условие выборки данных. Каждая строка таблицы, полученной в секции FROM, проверяется на соответствие условию независимо от других строк. В результате выполнения секции WHERE из результирующего набора данных удаляются строки, для которых не выполняется условие выборки данных. Если секция WHERE не задана, все полученные на предыдущем этапе строки сохраняются.

Секция GROUP BY. Секция GROUP BY группирует выбранный набор данных для получения набора сводных строк по значениям одного или нескольких столбцов. В одну группу объединяются строки набора данных с одинаковыми значениями полей, столбцы которых перечислены в секции GROUP BY. В результирующий набор данных заносится по одной строке для каждой группы.

Секция HAVING. В секции HAVING указывается условие выборки результата группировки, указанной в секции GROUP BY. Каждая группа, проверяется на соответствие условию независимо от других групп. Условие секции HAVING обычно содержит агрегатные функции (будут рассмотрены в разделе «Агрегатные функции»). В результате выполнения секции HAVING из результирующего набора данных удаляются строки для групп, для которых не выполняется условие.

Секция SELECT. В секции SELECT содержится описание столбцов результирующей выборки данных в указанном порядке. Синтаксис описания столбца следующий:

```
column_value [[AS] column_name]
```

где column_value — выражение, определяющее значение поля, column_name — имя столбца.

Выражение, в соответствии с которым формируются значения поля столбца, может содержать константные значения, имена столбцов таблицы из секции FROM, вызовы функций, значения которых будут вычисляться для каждой записи, агрегатные функции. Если имя столбца содержится в нескольких таблицах или табличных выражениях в секции FROM, перед именем столбца обязательно указывать имя или псевдоним соответствующей таблицы или табличного выражения.

Если требуется включить все столбцы результирующей выборки, можно использовать символ «*». Если требуется включить все поля только одной таблицы или табличного выражения из секции FROM, то перед символом «*» нужно указать имя или псевдоним соответствующей таблицы или табличного выражения.

Если в column_value указано имя столбца таблицы из секции FROM, это же имя будет назначено данному столбцу в результирующем наборе данных, а column_name позволяет задать столбцу другое имя. В остальных случаях, если не указано column_name, столбец не имеет имени.

Имена столбцов результирующей выборки данных могут повторяться. Если требуется обеспечить их уникальность, необходимо задавать для столбцов уникальные значения column_name.

Если в секции SELECT указано ключевое слово DISTINCT, из результирующей выборки данных удаляются все одинаковые строки. Далее выполняется секция ORDER BY. Если указана секция TOP, из результирующей выборки данных остаётся выбираются не более top_N строк, а все остальные строки удаляются.

Секция ORDER BY. В секции ORDER BY указываются столбцы секции SELECT или FROM, по которым выполняется сортировка. Если указано несколько столбцов, то сначала выполняется сортировка по первому столбцу, затем группы записей с одинаковыми значениями соответствующих полей упорядочиваются по значениям второго поля, затем группы записей с одинаковыми значениями полей первых двух столбцов упорядочиваются по значениям третьего столбца и так далее. Для каждого столбца указывается тип сортировки: ASC — по не-убыванию значений, DESC — по невозрастанию значений. Если тип не указан, используется значение ASC.

Агрегатная функция — это статистическая функция, которая принимает в качестве входного параметра имя столбца или выражения, выполняет вычисление на наборе значений и возвращает одиночное значение в качестве результата.

В языке SQL определены различные агрегатные функции. Из основных выделяют, например, COUNT, SUM, AVG, MIN и так далее. Более подробную информацию об агрегатных функциях изучите самостоятельно.

Вложенные подзапросы. Один SELECT-запрос может быть помещён в другой SELECT-запрос. В этом случае первый запрос является вложенным, а второй — главным. Вложенные подзапросы заключаются в круглые скобки.

Вложенные подзапросы можно разделить на три типа:

1) скалярные подзапросы — возвращают одно значение, могут использоваться в любом месте главного запроса, где ожидается простое значение соответствующего типа данных;

2) векторные подзапросы — возвращают табличное выражение, которое состоит из одного столбца. Его называют вектором данных. Векторные подзапросы могут использоваться в операции IN, которая проверяет наличие значения в массиве (векторе) данных;

3) табличные подзапросы — возвращают табличное выражение, содержащее произвольное количество строк и столбцов. Могут использоваться в любом месте главного запроса, где ожидается таблица или табличное выражение.

Столбцы табличного подзапроса должны иметь уникальные имена, тип столбцов определяется по значениям полей записей. Вложенные подзапросы могут использовать столбцы таблиц из главного запроса. Доступ к ним осуществляется указанием во вложенном подзапросе имён или псевдонимов таблиц главного запроса.

Объединение результатов нескольких SELECT-запросов. В некоторых случаях требуется выбрать данные из разных таблиц или других источников данных, имеющих одинаковую структуру, и объединить все строки полученных результатов в один результирующий набор данных. В этом случае записи можно рассматривать как элементы множеств.

Из двух SELECT-запросов можно получить один результирующий набор данных, указав между ними одну из следующих операций: UNION, UNION ALL, EXCEPT, INTERSECT.

Результатом объединения запросов является табличное выражение, которое также может участвовать в другой операции объединения результатов SELECT-запросов. Поскольку в этом случае не используются вложенные подзапросы, SELECT-запросы не заключаются в круглые скобки.

Задание к работе

1. Составить не менее трех различных SELECT-запросов к своей базе данных, созданной в результате выполнения предыдущих лабораторных работ, которые будут содержать объединение таблиц, агрегатные функции, а также группировку данных.
2. Выполнить задания по вариантам.

Вариант 1. Получить рейтинг (ТОП-5) наиболее популярных команд за сезон. Вывести рейтинг и эффективность игроков за заданный период.

Вариант 2. Получить рейтинг популярности товаров на основе предпочтений покупателей, в зависимости от различных групп товаров. Рассчитать сумму среднего чека по магазину в разное время суток.

Вариант 3. Получить среднее время простоя для тех водителей компании, которые совершали рейсы в заданную дату. Рассчитать соотношение доходности междугородних и местных доставок.

Вариант 4. Вывести наиболее часто заказываемые книги, сгруппированные по жанрам. Получить средний чек по разным группам товаров.

Вариант 5. Получить рейтинг продаж для продавцов-консультантов в группировке по различным моделям автомобилей. Получить список рекомендованных к заказу автомобилей на основании имеющегося запаса и популярности модели.

Вариант 6. Вывести рейтинг востребованности товаров в разрезе различных групп. Получить рейтинг дефицитности товаров в группировке по различным складам.

Вариант 7. Получить список рекомендаций для читателя с учетом его пола, возраста и прочитанных книг на основании общей статистики. Вывести среднее количество книг, прочитанных людьми за заданный период

Вариант 8. Составить упорядоченный список неплательщиков по возрастанию суммы долга в разрезе различных энергетических ресурсов. Получить рейтинг исполнителей работ за заданный промежуток времени (упорядочить по количеству отработанных заявок).

Вариант 9. Составить рейтинг убыточности товаров (наиболее часто требующих ремонта). Отчет по обеспеченности города мастерскими по разным группам товаров и всему ассортименту в целом.

Вариант 10. Подсчитать количество должников по подразделениям. Получить количественную оценку уровня острепенности преподавателей по подразделениям.

Вариант 11. Получить количество новых пользователей, зарегистрировавшихся, а также количество пользователей, удаливших аккаунт за указанный период. Подсчитать количество сообщений и комментариев в разрезе тем форума.

Вариант 12. Получить объем выплат по различным типам страховых случаев за период. Составить рейтинг менеджеров (Топ-5) компании в разрезе заключенных контрактов.

Вариант 13. Получить доходность гостиницы в зависимости от сезонности. Оценить актуальность предоставляемых услуг, в разрезе количественных показателей по разным группам услуг.

Вариант 14. Получить наиболее популярные направления (по количеству перевезенных пассажиров) в различные времена года. Посчитать рейтинг поездов (по количеству проданных билетов) в группировке по типам вагонов.

Вариант 15. Отчет по результатам предоставляемых услуг. Получить статистику пролеченных случаев заболеваний за заданный период.

Пример выполнения задания

Вывести топ три компании с максимальным бюджетом

```
SELECT executor.id, contact.fcs, budget
FROM contract
JOIN executor on executor.id = contract.executor_id
JOIN contact on contact.id = executor.contact_id
GROUP BY executor.id
ORDER BY budget DESC
LIMIT 3;
```

Вывести ФИО исполнителей в обратном алфавитном порядке, чьи работы завершены к текущей дате

```
SELECT contact.fcs
FROM contract
JOIN executor on executor.id = contract.customer_id
JOIN contact on contact.id = executor.contact_id
WHERE end_date <= CURRENT_DATE
GROUP BY contact.id
ORDER BY contact.fcs DESC;
```

Вывести ФИО заказчиков в обратном алфавитном порядке, чьи договора находятся на стадии исполнения

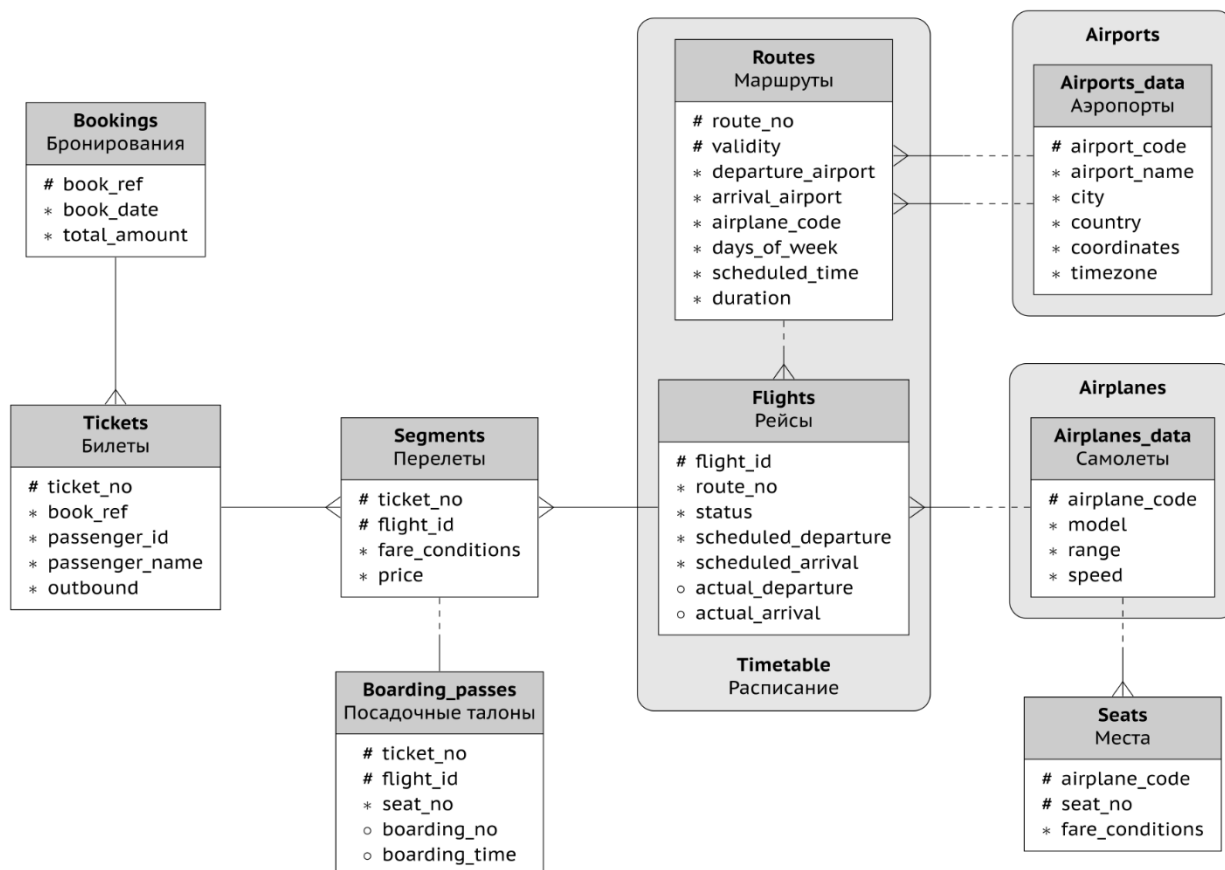
```
SELECT contact.fcs
FROM contract
JOIN customer on contract.customer_id = customer.id
JOIN contact on contact.id = customer.contact_id
WHERE end_date > CURRENT_DATE
GROUP BY contact.id
ORDER BY contact.fcs DESC;
```

3. Развернуть учебную базу данных Авиаперевозки и выполнить по ней построение sql-запросов согласно варианту. База данных доступна по адресу – <https://edu.postgrespro.ru/demo-20250901-3m.sql.gz>. Для развертывания базы данных можно выполнить следующую команду от имени пользователя *postgres* с использованием утилиты *psql*:

gunzip -c demo-20250901-3m.sql.gz | psql -U postgres.

Минимальная поддерживаемая версия — PostgreSQL 15.

Схема данных:



Описание схемы данных:

Основной сущностью является бронирование (*bookings*).

В одно бронирование можно включить нескольких пассажиров, каждому из которых выписывается билет (*tickets*) на поездку в прямом направлении, а также отдельный обратный билет, если билеты приобретаются «туда и обратно». Билет имеет уникальный номер и содержит информацию о пассажире. Для пассажира нет отдельной сущности, но можно считать, что номер документа уникально идентифицирует пассажира.

Билет включает один перелет (*segment*), если между пунктами отправления и назначения есть прямой рейс, и несколько перелетов, если

требуются пересадки. Все билеты в одном бронировании имеют одинаковый набор перелетов, хотя в схеме данных такого ограничения нет.

Маршруты (*routes*) проложены из одного аэропорта (*airports*) в другой. Каждый рейс (*flights*) следует по определенному маршруту, так что рейсы с одним номером маршрута имеют одинаковые пункты вылета и назначения, но отличаются датой отправления.

Маршрутная сеть меняется раз в месяц, поэтому рейсы и маршруты связаны темпоральным внешним ключом: при соединении следует учитывать не только номер маршрута, но и соответствие времени отправления интервалу действия маршрута. Для упрощения запросов можно использовать представление *timetable*, скрывающее соединение этих таблиц.

Все перелеты считаются стыковочными: пассажир проходит регистрацию на первый рейс и сразу получает посадочные талоны (*boarding_passes*) на все рейсы в своем билете. В посадочном талоне указано место, а также фиксируется время посадки в самолёт. Пассажир может зарегистрироваться только на тот рейс, который есть у него в билете. Комбинация рейса и места в самолёте уникальна, чтобы не допустить выдачу двух посадочных талонов на одно место.

Количество мест (*seats*) в самолёте и их распределение по классам обслуживания зависит от модели самолёта (*airplanes*), выполняющего рейс. Предполагается, что каждая модель самолёта имеет только одну компоновку салона. Схема данных не контролирует, что места в посадочных талонах соответствуют имеющимся в самолёте (такая проверка может быть сделана с использованием табличных триггеров или в приложении).

Вариант 1

1. Найдите все аэропорты, расположенные в России.
2. Найти все рейсы, вылетающие из аэропорта 'SVO' (Шереметьево) в ближайшие 24 часа от текущего времени среза базы данных (функция `bookings.now()`). Вывести номер рейса, время вылета по расписанию и статус.
3. Найдите аэропорты, из которых вылетает более 100 различных рейсов (уникальных *flight_id*) в текущем срезе данных. Выведите код аэропорта, название и количество рейсов.

Вариант 2

1. Выведите список всех моделей самолётов с их максимальной дальностью полёта, отсортированный по убыванию дальности.
2. Вывести топ-5 самых дорогих бронирований. Для каждого бронирования показать его номер, дату оформления и общую сумму.

3. Для каждого бронирования, включающего более одного билета, выведите номер бронирования, общую сумму (*total_amount*) и среднюю стоимость одного билета в этом бронировании.

Вариант 3

1. Сколько всего бронирований было сделано в базе?
2. Найти всех пассажиров с фамилией 'Ivanov' (учтите, что имя и фамилия хранятся в одном поле *passenger_name*). Вывести номер билета, номер бронирования и полное имя пассажира.
3. Найдите модели самолётов, на которых выполняется более 50 рейсов. Выведите код самолёта, модель и количество рейсов.

Вариант 4

1. Найдите все рейсы, вылетающие из аэропорта DME (Домодедово) и прибывающие в аэропорт SVO (Шереметьево).
2. Посчитать общее количество проданных билетов в бизнес-классе ('Business') по всем рейсам.
3. Выведите список пассажиров (*имя* и *ID*), у которых общая стоимость всех перелётов (сумма *price* по всем их билетам) превышает 100 000 рублей.

Вариант 5

1. Выведите номера билетов и имена пассажиров, у которых есть билет «туда и обратно» (т.е. в одном бронировании есть и *outbound = true*, и *outbound = false* для одного *passenger_id*).
2. Найти рейсы, которые были отменены ('Cancelled'). Вывести номер маршрута, запланированное время вылета и фактическое время вылета (если оно есть).
3. Найдите аэропорты назначения, в которые прибывает более 200 рейсов. Выведите код аэропорта, город и количество прибывающих рейсов.

Вариант 6

1. Сколько мест в самолёте модели 'SU9' (код ИАТА)?
2. Для каждого аэропорта вылета посчитать количество вылетевших из него рейсов. Отсортировать результат по убыванию количества рейсов.
3. Для каждого класса обслуживания (*fare_conditions*) определите среднюю стоимость перелёта и количество таких перелётов, но только если их не менее 1000.

Вариант 7

1. Найдите все рейсы со статусом 'Delayed', которые вылетают сегодня (по локальному времени вылета). Используйте функцию `bookings.now()` для определения «сегодня».
2. Найти модели самолетов, дальность полета которых превышает 5000 км. Вывести код самолета, название модели и дальность полета.
3. Найдите бронирования, в которых все билеты относятся к одному и тому же пассажиру (т.е. только один *passenger_id* в бронировании), но при этом включают как прямой, так и обратный билеты. Выведите номер бронирования и имя пассажира.

Вариант 8

1. Выведите общую сумму всех бронирований, сделанных в сентябре 2024 года.
2. Вывести список пассажиров (имя и номер документа), у которых есть билеты "туда и обратно" (т.е. в одном бронировании для одного пассажира есть два билета с разными значениями *outbound*).
3. Определите аэропорты, из которых вылетают рейсы только в один и тот же аэропорт назначения. Выведите код вылета, код прилёта и название аэропорта вылета.

Вариант 9

1. Найдите пассажиров, у которых есть посадочные талоны на рейс с *flight_id = 12345*. Выведите их имена и номера мест.
2. Найти все посадочные талоны, выданные на рейс с *flight_id = 12345*. Вывести номер билета, номер места и номер посадочного талона.
3. Найдите модели самолётов, у которых все рейсы имеют статус 'Arrived'. Выведите код и модель самолёта.

Вариант 10

1. Сколько перелётов (*segments*) включено в бронирование с *book_ref = 'ABC123'*.
2. Посчитать среднюю стоимость перелета (*segments.price*) для каждого класса обслуживания (*fare_conditions*).
3. Выведите номера бронирований, в которых все перелёты стоят дороже 10 000 рублей.

Вариант 11

1. Выведите список всех рейсов, выполняемых самолётом '733', с указанием аэропортов вылета и прилёта.

2. Найти бронирования, в которых больше одного пассажира. Вывести номер бронирования и количество пассажиров в нем.
3. Найдите аэропорты, из которых вылетают рейсы минимум в 10 разных городов (учитывая город аэропорта назначения). Выведите код аэропорта и количество городов.

Вариант 12

1. Найдите все билеты, стоимость которых превышает 50 000 рублей. Выведите номер билета, имя пассажира и класс обслуживания.
2. Вывести расписание рейсов на сегодня (от *bookings.now()*) для аэропорта 'LED' (Пулково), используя представление *timetable*. Показать номер рейса, аэропорт назначения и местное время вылета.
3. Определите пассажиров, у которых все перелёты оформлены в классе 'Business'. Выведите их *ID* и *имя*.

Вариант 13

1. Сколько пассажиров зарегистрировались на рейс с *flight_id* = 67890 (т.е. имеют посадочный талон).
2. Найти рейсы, у которых фактическое время прибытия (*actual_arrival*) превысило запланированное (*scheduled_arrival*) более чем на 2 часа. Вывести номер рейса и разницу во времени.
3. Найдите бронирования, в которых общая сумма (*total_amount*) не совпадает с суммой всех перелётов в билетах. Выведите номер бронирования, сумму из *bookings* и реальную сумму перелётов.

Вариант 14

1. Выведите аэропорты, из которых нет вылетающих рейсов (в текущем срезе данных).
2. Для каждого самолета (модели) посчитать общее количество мест в салоне, а также количество мест в бизнес-классе.
3. Выведите коды аэропортов, из которых вылетают рейсы только в пределах одной страны (страна прилёта совпадает со страной вылета).

Вариант 15

1. Найдите все бронирования, в которых участвует более одного пассажира. Выведите номер бронирования и количество пассажиров.
2. Найдите модели самолётов, у которых средняя загрузка (количество выданных посадочных талонов на рейс) превышает 80% от общего числа мест в салоне. Выведите код самолёта, модель и среднюю загрузку (%).

3. Найти пассажиров, которые летали на одном и том же рейсе (выберите любой конкретный *flight_id*, например, 67890). Вывести их имена и номера билетов.

В процессе выполнения лабораторной работы были изучены основные принципы создания SQL-запросов для выборки данных из таблиц базы данных и представления данных в требуемом виде.